



END OF STUDIES INTERNSHIP REPORT

EPITECH-Promotion 2026

Design and development of a bank reconciliation platform

Student	Tom DUMORT
EPITECH email	tom.dumort@epitech.eu
Host company	Reveals -Luxembourg
Internship period	From 01/04/2026 to 30/09/2026
Company tutor	Florian Lamant - Developer

Acknowledgements

I want to thank Florian Lamant, my tutor at Reveals, for his support, his availability and the trust he gave me during all this internship. I also thank all the Reveals team for their welcome and their support, and the EPITECH teaching team for the follow-up during this end of studies period.

Table of contents

Acknowledgements	2
General introduction	3
1. The Reveals company	5
2. The client project: bank reconciliation platform	5
3. General architecture of the solution	6
4. Organization of the work and the team.....	7
5. Key business concepts	7
6. Difficulties encountered	8
7. Conclusion of part 1	9
Strengths.....	11
Areas for improvement.....	11
Context	13
The project.....	13
Interest for the company	13
Organization and follow-up of the project	13

General introduction

Reveals is a company specialized in developing tools in the data management and banking area. So it's in this context that I worked during the 6 months of my end of studies internship. During these 6 months I was placed on 2 different projects. The first one, a software allowing a company that does regulatory controls to organize their control plan. The second one a bank reconciliation software for a Luxembourgish bank that just finished their new banking system.

In the first part of this report, we will focus only on this second project that I led from the start to the end and that I find more relevant to present to you.

This report will then be in 3 parts, first the presentation of the project, its stakes, the progress of the project and the steps to follow to take over the project efficiently, then we will focus the second part of this report on an end of internship status, to finish with a 3rd part that will be about a project that would mean a lot to me to develop if the opportunity came up.



PART 1

Context, architecture and organization of the project

As a reminder, this part will be dedicated to a quick presentation of the company, the work environment and then the description of the project I was placed on during my internship. Since this part is supposed to be presented to someone new in the company, the tone of this part can sometimes contain technical vocabulary. They will however be explained at the end of the part to help the general understanding

1. The Reveals company

Hello, first of all welcome to Reveals, this company was launched around 5 years ago by David Recchia with the goal to serve companies the best way possible in the management and processing of their data. Today we are around fifteen people working here and even so the atmosphere stays quite familial which gives a rather nice work environment. Here most of the employees are deployed at clients to be able to answer their needs the best way, but also from the office when the projects allow it. Most of our clients come from the banking world because it's our specialty, our goal on a project is not just to deliver a working product but also to try to bring our expertise to improve the final quality of the product. Here you never stay stuck alone on a project for very long, if you need help don't hesitate someone will come help you without problem.

2. The client project: bank reconciliation platform

Let's talk now about the project you will work on starting tomorrow: the Bank reconciliation tool for the new CBS of this bank. We will go step by step. First of all what is a bank reconciliation tool and why it is mandatory when a new CBS goes into production.

To understand all this you first need to understand how a banking payment system works. When a user makes a transaction, the money is not directly sent from your account to the recipient's one, it has to go through several intermediary accounts called float or transit accounts. The goal of the reconciliation tool is to make sure the money moves correctly from one account to another, which makes sure that the money that left is 1) really left, 2) left to the right place and 3) really arrived. This is why this kind of tool is mandatory in banks, because being subject to very strict regulations on money movements, they cannot afford to have money that «wanders» outside of their control.

So we have several money flows to reconcile, namely the ATM flow, WebRipost, incoming and outgoing IP, non-IP incoming and outgoing flow and Nostro Mirror IP flow. Each of these flows has its use in the loop but also has a different way of working depending on the flow.

To give you an idea, the non-IP incoming and outgoing flows already represent 2 million money movements, around 1 million for the IP flows, 500 thousand for ATM and WebRipost and 1 million for Nostro Mirror IP, all that in a very short time, just so you realize the volume of data to process. Also Wero is coming to Luxembourg and will need to be integrated into the application very soon, you might get the chance to work on it.

The application also has a complete traceability system for user actions as well as a dynamic access management depending on the users.

The application runs permanently on the bank's servers so users can access it at any time, but the reconciliation task itself only runs once a day.

3. General architecture of the solution

Let's move now to a bit more technical point: how the application is actually built. I will try to keep it simple, some terms will be explained again a bit later in this part anyway.

The architecture is based on three main building blocks: an API that centralizes all the business logic and the data access, a PostgreSQL database that stores all the movements and reconciliations, and a web interface in Vue.js where the operators come check and handle the cases every day. Around these three blocks comes Airflow, the orchestrator that runs all the automatic processes: getting the data from the CBS and the different source files, loading it into the database, then starting the reconciliation.

The general working can be summarized like this:



Simplified diagram of the processing flow of a movement, from its reception to its archiving.

In practice, the files and extractions come from the different sources (CBS, ATM files, WebRipost...), they are picked up and loaded into the database by the scheduled Airflow processes, and each movement is then stored in a table called «active» while waiting to be processed. Once a day, the reconciliation engine groups these movements by flow and by reconciliation key, and as soon as a group is balanced (the sum of the amounts falls to zero), it is validated automatically. The processed movements then move to a second table called «archive», which keeps the active table light and fast despite the millions of movements going through it every month.

To make things easier to find, the database is also split into several different schemas: one for everything related to the application configuration (users, rights, flows...), one for everything specific to the reconciliation business (movements, groups, exclusions...), and a last one dedicated to traceability, which keeps a trace of every action, whether it is done automatically by the system or manually by a user.

One last point on the architecture before moving on: each flow has its own parser, adapted to the format it receives. For ATM, the files come in a fixed-width Cobol format, so you have to read exact positions in each line to extract the fields. For WebRipost, these are Excel files. Other flows are driven by a configuration (delimiter, column mapping...) that allows reading CSV or XML without rewriting a parser every time. The SWIFT MT940 bank statements also have their own dedicated parser. For the flows that come directly from the CBS however, there is not even a file: the application connects directly to the database and looks for the new movements with a query, with a cursor that remembers how far it already went last time. All these parsers still follow the same common interface, so that if one day a new flow with a still different format needs to be added, it is enough to write a new parser that respects it, without touching the rest of the processing chain.

There is also a nuance to understand between the ways the flows coming directly from the CBS work, in fact there are no less than 3 different ways to aggregate the data and 8 methods to get the information depending on the composition of certain fields. These 3 ways were however different by flow and break down as follows:

- single movements (1-1) (except exceptions)
- bulk movements (1-N) for the non IP inward flow
- batch movements (N-N) for the non IP outward flow

As announced at the start of this part, the concept is actually very simple but in practice it needs a lot of investment to really understand all the CBS workings.

4. Organization of the work and the team

For this project we worked as a team of 3, to work efficiently we used Git and Github for the collaborative work, about the organization, a short 10 min meeting every day / every 2 days, as well as a meeting with all the teams every week to sync everyone and reassign resources where needed. When an incident is reported by a tester or a user, they create a jira ticket, our job is also to take care of each of these tickets in parallel of the application developments. On this point, it's a very basic way of working.

5. Key business concepts

Before continuing, I will go over here a few terms that come back often when talking about the tool, so that the rest of the report, and especially your daily life once on the project, is easier to follow.

Movement: it's the base building block of the tool, a line that represents a transaction on a transit or float account of the bank. Each movement belongs to a flow and carries an amount, a value date and a status.

Flow: the channel a movement belongs to, ATM, WebRipost, incoming IP, outgoing IP, etc. Each flow has its own processing rules and its own way to compute its reconciliation key.

ATM: the movements linked to cash machines, withdrawals as well as deposits. The flow compares what is booked on the bank side with the files sent by the cash machine network, in that famous fixed-width Cobol format I talked about earlier.

WebRipost: the operations made through the partner points of sale network, cash deposits and withdrawals, cheque deposits. They come as a CSV or Excel export from an external system called Webripost

Nostro Mirror IP: how it works and what it is: a bank that sends or receives money internationally usually goes through an account it holds at another bank, called a «nostro» account. The «mirror» here means the copy of what is booked on the bank side for the instant payments (IP) that go through this account, to compare with the SWIFT MT940 bank statements received back to check that everything matches.

Reconciliation key: the criteria used to group together the movements that are supposed to compensate each other. It's what allows the tool to know which movements to compare with each other.

Reconciliation group: all the movements that share the same key. As soon as the sum of the group's amounts falls to zero, the group is validated automatically.

Forcing: when a group does not balance by itself but a user thinks the movements are really linked, they can force the reconciliation manually, as long as they leave a comment that justifies their decision.

Exclusion: on the opposite, an isolated movement can be removed for good from the automatic reconciliation circuit, with a mandatory reason, to avoid it getting associated with the wrong movement later.

Émargement: once a movement is processed, if it's matched, forced or excluded, it is moved to the archive table I was talking about in the previous part. We say it is «émargé» (archived). This is what allows to keep the active table light despite the volume.

Batch processing: on some flows, a single bank movement can actually group several individual payments, for example a grouped transfer that settles several operations at once. We had to build a specific process to rebuild these payments before being able to reconcile them one by one, this is actually one of the points I detail in the next part.

Single processing (IP): simple instant movement coming from a single person and going to a single person too.

Bulk processing: on some flows you can find a system similar to batch processing but in a simplified way. To make it simple, instead of having incoming movements that can be spread into outgoing movements and the other way around, here we simply have one big incoming movement and a lot of outgoing movements.

6. Difficulties encountered

The first big difficulty came from the flow that handles the grouped movements, the ones where a single bank transfer actually settles several payments at once. At the very beginning, the algorithm that rebuilt these groupings would sometimes get carried away: a single generic label, shared by dozens of payments with no relation between them, was enough to stick everything together. We ended up with a single lot of more than 50 000 movements, which obviously could never balance and blocked, on the way, the few hundred payments that should have matched normally. We had to review in depth the way these groupings were built so that each lot stays linked to what it really concerns, without getting polluted by labels that are too generic. The lesson I take from it is that a grouping key that looks discriminant on a sample can very well not be one at all anymore at the scale of several months of data.

The second difficulty is about the way we rebuild the individual payments inside a grouped movement. At first, when the same amount had to be split between several payments, we did it on a pro rata basis. On paper it works, but at a big scale the roundings add up: we ended up with several hundred lots that never balanced completely, by one cent. It looks like almost nothing but it's enough for a lot to stay pending forever. The fix was to stop splitting on a pro rata basis and instead directly compute the exact

amount each group has to carry. It taught me to be careful with proportional splits as soon as money is involved: it's better to compute an exact amount than to deduce a share from it, even a tiny one.

We also had an incident in production on the database. The automatic reconciliation runs once a day and moves the processed movements to the archive table while another part of the process updates their status, on these same movements. One day, these two operations both tried to lock the same rows at the same time, in a different order, which PostgreSQL detects as a deadlock and refuses by cutting one of the two transactions. The job of the day failed. We had to review the execution timeouts of this process, which was initially thought as a simple API call while it's actually a background process on several million rows. It's the kind of incident you never see in a test environment with a low volume, but that becomes almost inevitable once in production with real volumes.

One last difficulty, more classic: when trying to deactivate or change the configuration of a flow that was not used anymore, the application refused the operation with an integrity constraint error. By digging in, we realized that lots already created still referenced the old configuration of this flow, and that nothing had planned for this case when it was deactivated. Nothing serious by itself, but it reminded me that a system that has been living for several months builds up cross references that you don't necessarily think about when you write a feature, and that you need to think about the whole life cycle of an object (creation, update, deactivation, deletion) and not only its main use case.

Today, all these problems are under control or mostly already fixed, the main problem that still causes big issues and that lowers the scores of the reconciliation application is the stability of the CBS. This is actually what takes most of our time lately: trying to help the teams involved to stabilize the tool.

7. Conclusion of part 1

So, you now have an overview of what you are arriving on: who Reveals is, why this reconciliation tool exists, how it is built, how we work on it every day, the vocabulary we use between us, and the main issues we already met along the way. The good news is that most of these difficulties are behind us today, fixed or under control. The real work right now is more on the CBS side: it's a system that is still young and not always stable, and a good part of our time lately consists exactly in helping the teams involved to stabilize it rather than developing new features on the reconciliation tool itself.

This will probably also be your daily life when you arrive: as much time spent understanding why a movement does not arrive as expected on the CBS side as coding on the application itself. Take the time to really digest this part before jumping into the code, a good part of the issues we had actually came from a CBS behavior that was not anticipated at the start. For the rest, the team is small and easy to reach, never hesitate to ask.



PART 2

Personal end of internship review

This part writes down the arguments I want to present to my tutor / manager at the end of the internship: my strengths and my areas for improvement

Strengths

Thinking back on these six months, there are three things that I think really made the difference. The first one is my speed of adaptation and understanding. Arriving on a project that is already this established, with its own vocabulary and its own business subtleties, needs to get skilled fast, and I think the path I followed at EPITECH clearly helped me on this point: there you learn to manage alone in front of a subject you don't master yet. The second one is my determination at work. I always volunteered as soon as a slightly more complex or less comfortable subject came up, instead of waiting for it to be given to me. This involvement was actually recognized internally: the boss congratulated me directly for it, and I got an end of internship bonus for the efforts made, which touched me a lot. The third point is that I socialize easily with people. It allowed me to integrate very quickly into the team, but also to build a real trust relationship with the people on the client side, which matters a lot on a project where you spend as much time talking as coding.

Areas for improvement

On the points to work on, I would say mostly my ability to keep my attention focused during discussions. It happens that I get distracted or start thinking about something else while someone is explaining something to me, which sometimes forces me to ask for a question to be repeated or to reformulate what I just understood. It's not a problem of understanding by itself, more a question of listening discipline, and it's an area I'm consciously working on right now, notably by taking notes more systematically during discussions.



PART 3

The project of my dreams

This last part changes tone. We are not presenting something existing anymore but making a pitch: here I imagine talking directly to the Reveals management to defend a project I would like to lead if the opportunity came up. The idea comes from a very concrete observation made during this internship.

Context

Over the last months, the difficulty that took most of our attention was not really the reconciliation tool itself anymore, but the stability of the CBS it relies on. We spend a good part of our time understanding afterwards why a movement never arrived as expected, instead of knowing it before it becomes a problem. This observation, repeated week after week, made me want to tackle this point directly instead of suffering its consequences on the reconciliation side.

The project

The idea would be to build a CBS monitoring tool, separate from the current reconciliation tool but relying on the same data flows. In practice, instead of waiting for missing or inconsistent movements to show up through reconciliation, the tool would continuously monitor a few simple indicators on the CBS side (expected volumes per flow, processing delays, repeated errors on the same movement type) and raise an alert as soon as a behavior goes out of the normal range. Without meaning to, we built up during this internship a good knowledge of the signals that announce a problem before it explodes on the reconciliation side. This project would exactly be about formalizing this intuition instead of keeping it in our heads.

Interest for the company

For Reveals, the interest is direct: today we spend a significant amount of time reacting to CBS incidents instead of building value on the reconciliation tool. Detecting things upstream means time given back to higher value tasks, and it's also a concrete commercial argument to present to the client: we don't just notice the problems, we anticipate them. A tool like this could actually be offered as it is to other Reveals clients facing the same kind of instability on their banking systems.

Organization and follow-up of the project

I would see this project start small: a first milestone of a few weeks to identify, with the CBS teams, the most telling signals of the incidents already encountered, before building anything. Then a prototype limited to a single flow to validate the approach before generalizing it. I would suggest following the progress with a simple indicator: the number of incidents detected upstream against the number of incidents suffered, month after month, to concretely prove the value of the tool instead of just staying a promise.